

Generative AI Engineering

From Foundations to Agentic AI Systems — A 12-Week Industry-Ready Program

Duration: 12 Weeks (3 Months)	Total Volume: 132 Contact Hours	Format: Theory + Hands-on Labs + Capstone	Level: Foundations to Agentic AI
-------------------------------	---------------------------------	---	----------------------------------

Program Overview

This program takes students from data handling and backend engineering through classical NLP, deep sequence models, Transformers, and Retrieval-Augmented Generation, into LLMOps and Agentic AI. Rather than a set of disconnected exercises, the course follows a **single progressive build**: each week adds a new capability to a system the student carries forward, so that by the end of the program every student has a working, evaluated, deployed AI application.

The Industry Learning Cycle: Every topic follows the same rigorous learning cycle: **Concept** → **Architecture** → **Demonstration** → **Build** → **Break** → **Debug** → **Evaluate**. This ensures students don't just call an API — they understand why each architecture works and can defend their design choices in an interview.

Program Structure & Curriculum Breakdown

Component	Hours	Purpose & Focus
Theory / Concepts	66 h	Understand why each architecture and system actually works
Hands-on Labs	44 h	Build and test individual modular pipeline and modeling components
Projects / Capstone	18 h	Integrate everything into one deployed, evaluated production system
Evaluation / Defense	4 h	Benchmark, test against regressions, and present/defend the final system

Technical Prerequisites

Required (Pre-Course)	Not Required (We Teach These)
• Python (OOP & functional patterns) • Linear Algebra & Calculus (applied to ML) • Statistics for AI • Git & Linux fundamentals • Pandas / NumPy basics	• Prior ML / Deep Learning experience • Cloud certification • Advanced distributed systems • Research publication experience

* Note: Students without a programming background can take an optional 8–10 hour Python, API, and Git bridge module before the program begins.

Week-by-Week Curriculum (Weeks 1 to 12)

Week	Module & Title	Key Concepts	Hands-on Practical Work
Week 1	Data Engineering for AI	Handling large/messy datasets, EDA strategy, data quality issues, Git workflows in team AI projects.	Clean and profile a large real dataset with Pandas/NumPy; Git branching + PR workflow; Linux CLI drills.
Week 2	AI Backend Engineering I	REST API design for ML/LLM serving, FastAPI vs Flask, request/response patterns for model inference.	Build a FastAPI service that serves a model endpoint; add request validation and error handling.
Week 3	AI Backend Engineering II	PostgreSQL fundamentals, Redis caching for AI apps, Docker, basic system design (rate limiting, caching, URL shortener pattern), logging/monitoring.	Add Postgres + Redis to the Week 2 service; containerize with Docker; implement a rate limiter. Output: Dockerized AI backend — the industry-readiness gate before NLP/LLMs.
Week 4	NLP Foundations: Text Representation	NLP challenges, tokenization, stemming/lemmatization, One-Hot Encoding, Bag of Words, TF-IDF.	Build a text-cleaning pipeline with NLTK; implement BoW and TF-IDF from scratch and with sklearn.

Week 5	Word Embeddings & Sequence Models	Word2Vec (CBOW/Skip-gram), Average Word2Vec, ANN vs RNN, why sequence models need memory, word embeddings as feature representation.	<i>Train Word2Vec on a custom corpus; build RNN and LSTM classifiers; compare LSTM vs GRU vs BiRNN on the same task.</i>
Week 6	Seq2Seq & Attention	Encoder-Decoder architecture, Seq2Seq, the fixed-context-vector bottleneck, the attention mechanism (Bahdanau/Luong).	<i>Build a toy Encoder-Decoder Seq2Seq model; implement attention on top of it and demonstrate the improvement.</i>
Week 7	Transformers	Self-attention, multi-head attention, positional encoding, why Transformers replaced RNNs.	<i>Fine-tune a pretrained Transformer (BERT) for classification; complete the evolution benchmark: BoW → Word2Vec → LSTM → Attention → Transformer.</i>
Week 8	Embeddings, Vector Databases & Retrieval	Embedding spaces for retrieval, FAISS/Chroma internals, BM25 (sparse) vs dense retrieval, hybrid search.	<i>Build embedding pipelines with FAISS and Chroma; implement hybrid BM25 + dense retrieval and compare recall.</i>
Week 9	RAG, Prompt Engineering & LLM APIs	RAG architecture and chunking strategies, few-shot/eval-driven prompt engineering, LLM APIs (OpenAI/Azure), evaluation metrics (faithfulness, relevance, latency, cost).	<i>Build a full end-to-end RAG pipeline; build an evaluation dashboard tracking those metrics per query.</i>
Week 10	LangChain/LangGraph & LLMops	LangChain/LangGraph fundamentals, prompt/dataset versioning, experiment tracking, cost tracking, observability, guardrails, CI/CD for AI apps.	<i>Rebuild the RAG pipeline in LangChain/LangGraph; instrument it with versioning, observability, and a guardrail layer; set up a CI/CD eval-regression check.</i>
Week 11	Agentic AI & Cloud Deployment	Multi-step reasoning agents, tool-calling, memory, planning agents; AWS/Azure basics, Kubernetes fundamentals, scaling AI APIs.	<i>Build a tool-calling agent with memory using LangGraph; deploy the Week 9–10 system to cloud infra with autoscaling.</i>
Week 12	Capstone	Research methodology: problem framing, evaluation design, technical reporting.	<i>Student-chosen real problem (legal doc assistant, medical Q&A;., disaster-response assistant, internal knowledge bot). Design → Build → Evaluate → Deploy → Report, then present and defend.</i>

Capstone Project — End-to-End Production Agentic RAG System

Students tackle a student-chosen real problem (e.g. legal doc assistant, medical Q&A;., disaster-response assistant, or enterprise internal knowledge bot) and execute the complete lifecycle: **Design** → **Build** → **Evaluate** → **Deploy** → **Report**.

Capstone Deliverables & Requirements:

- **Agent / RAG System:** Integrated architecture combining hybrid retrieval, dynamic tool calling, and stateful conversational memory.
- **Evaluation Framework:** Telemetry logging of critical metrics including accuracy, token cost, query latency, and guardrail safety.
- **Containerized Cloud Deployment:** Deployed on cloud infrastructure with autoscaling and reverse-proxy load balancing.
- **Documented Improvement Cycle:** Explicit versioned evolution demonstrating measurable gains from Version 1 to Version 2.
- **Technical Defense:** A formal research/technical report defending architecture choices and design decisions before an expert panel.

Graduate Learning Outcomes

By the conclusion of this program, students will be verified capable to:

- | | |
|--|---|
| <ul style="list-style-type: none"> ✓ Build and containerize a production AI backend (FastAPI, Postgres, Redis, Docker) ✓ Explain and implement the NLP evolution from Bag-of-Words to Transformers ✓ Build sequence models (RNN, LSTM, GRU, BiRNN, Seq2Seq) and diagnose limits ✓ Implement attention mechanisms and fine-tune Transformer models (BERT) | <ul style="list-style-type: none"> ✓ Build and evaluate RAG systems using vector databases and hybrid retrieval ✓ Apply LLMops practices: versioning, observability, guardrails, CI/CD for AI ✓ Design and deploy agentic AI systems with tool-calling, memory, and planning ✓ Deploy AI systems to the cloud and defend a complete system end to end |
|--|---|